



AWS Project: 3D E-Commerce Store Architecture

INTRODUCTION

An ecommerce platform is software that enables business owners to set up and run an entire online store. Ecommerce platforms are important because they allow customers to purchase anytime and anywhere, are cost-effective by reducing expenditures such as rents and other utilities, enable businesses to easily scale operations, introduce new products, and adapt to changing market demands, and facilitate easy comparison of different products and prices, allowing customers to make better decisions.

The platform is broad and diverse, involving business model types such as Business-to-Business (B2B), Business-to-Consumer (B2C), and Consumer-to-Consumer (C2C). Other platform types include Software-as-a-Service (SaaS) – all tools are available online to run your store; Platform-as-a-Service (PaaS) – open-source that includes both software and hardware, allowing users to customize the software; and On-Premise platforms – where businesses manage their own servers and hardware within their physical premises, giving them full control over the infrastructure.

3D E-COMMERCE DESIGN

This architecture is designed to support a next-generation 3D e-commerce web application that allows millions of global users to interact with 3D models of products (e.g., furniture, gadgets, fashion items) in a fast, highly available, secure, and cost-efficient manner. It achieves high availability through multi-AZ deployments and automatic failover, scalability via auto-scaling and serverless components, performance with edge caching and low-latency databases, security via best practices like encryption and authentication, and cost optimization by using pay-per-use services and monitoring tools.

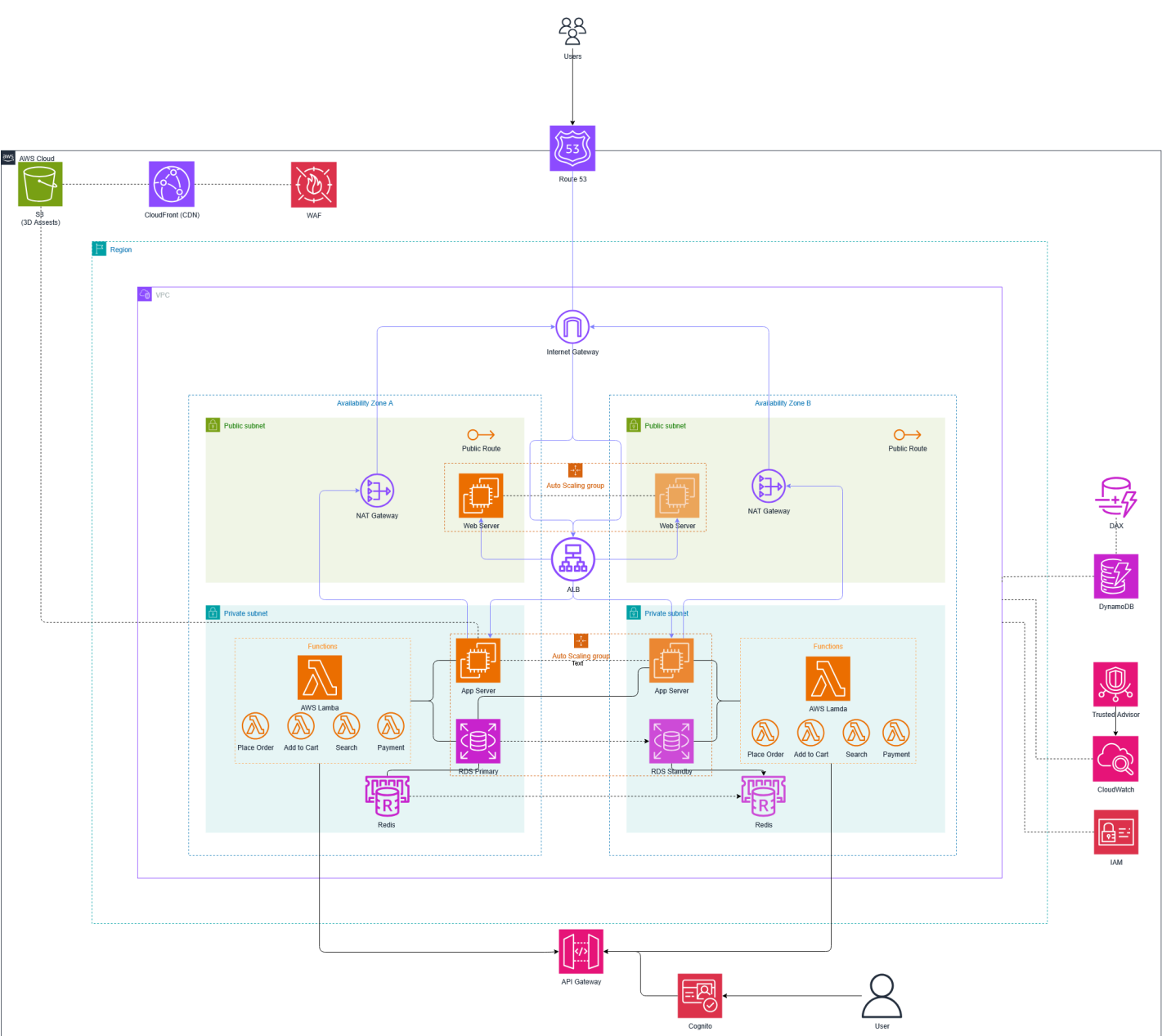
Here's a clear explanation of what the architecture does, where components are placed, and how they work together:

- **User Entry and Global Distribution (Presentation Layer - Outside VPC, Global):** Users access the platform via a web browser or app. Amazon Route 53 handles DNS resolution, routing traffic to the optimal endpoint based on latency or health checks (e.g., directing to the nearest CloudFront edge location). Amazon CloudFront, a global CDN, caches and delivers static content like HTML, CSS, JS, and 3D assets from Amazon S3 buckets, ensuring low-latency rendering worldwide (performance for millions of users). AWS WAF integrates with CloudFront to filter malicious traffic at the edge (security). How it works: A user request hits Route 53, which points to CloudFront; CloudFront pulls content from S3 (stored in a regional bucket) and caches it at over 200 edge locations, reducing load on origins.
- **Authentication and API Routing (Application Layer - Regional, Integrated with VPC):** After initial static load, dynamic interactions (e.g., login, search) go through Amazon API Gateway, which acts as a secure front door for backend APIs. Amazon Cognito handles user authentication (e.g., sign-up, login with JWT tokens),

integrating with API Gateway to authorize requests. Where: API Gateway and Cognito are managed services outside the VPC but connect securely. How: A authenticated request is routed by API Gateway to backend targets like AWS Lambda, with throttling and caching to handle spikes (scalability).

- **Backend Processing and Compute (Application Layer - VPC Private Subnets, Multi-AZ):** AWS Lambda functions (serverless) process core logic like "Search," "Add to Cart," "Place Order," and "Payment," auto-scaling to handle unpredictable traffic without provisioning (cost-efficient). For compute-intensive tasks like real-time 3D rendering, Amazon EC2 instances in an Auto Scaling Group (e.g., in private subnets) provide dedicated resources, distributed by an Application Load Balancer (ALB) in public subnets. Where: Lambdas can run VPC-enabled in private subnets for resource access; EC2 in private subnets across Availability Zones A and B. How: API Gateway invokes Lambdas directly; for EC2, traffic flows from API Gateway to ALB (via internet or private link), then to EC2 targets. NAT Gateways in public subnets allow private resources outbound internet access (e.g., for payment APIs).
- **Data Storage and Caching (Data Layer - Managed Services, Multi-AZ/Global):** Amazon DynamoDB stores dynamic, high-volume data like carts and orders with single-digit ms latency (performance), auto-scaling reads/writes (scalability). Amazon DAX provides in-memory caching for DynamoDB to further reduce latency. Amazon RDS (PostgreSQL, Multi-AZ) handles relational data like product catalogs and user profiles, with automatic failover to a standby in another AZ (high availability). Amazon ElastiCache (Redis) caches sessions and frequent queries in private subnets. Amazon S3 stores 3D assets durably. Where: DynamoDB, DAX, RDS, and ElastiCache are multi-AZ; S3 is regional but highly available. How: Lambdas/EC2 query databases via VPC Endpoints (private traffic, security/cost savings); data flows securely with IAM roles enforcing least privilege.
- **Networking and Security (Foundation - VPC, Regional):** The Amazon VPC isolates resources with public subnets (for ALB, NAT Gateways) and private subnets (for EC2, databases). Internet Gateway enables inbound public access, while VPC Endpoints keep S3/DynamoDB traffic internal. AWS IAM manages roles and policies for all components. How: Traffic enters via IGW to public components, routes internally; security groups and NACLs control access.
- **Monitoring and Optimization (Cross-Cutting - Regional/Global):** Amazon CloudWatch collects metrics, logs, and sets alarms across all services (e.g., triggering auto-scaling). AWS Trusted Advisor provides recommendations for cost, security, and performance. How: Integrated dashboards alert on issues; Trusted Advisor scans for optimizations like underutilized resources.

This top-down flow ensures 24/7 availability, handles spikes for millions of users, delivers fast 3D interactions, enforces security, and optimizes costs by avoiding over-provisioning.



AWS SERVICES ANALYSIS

Below is an updated analysis of the AWS services in the corrected architecture. I've added entries for newly emphasized components (e.g., DAX, ElastiCache) and removed irrelevant ones (e.g., Kinesis, as it's not core to this design). Each explains why chosen, how it meets the five requirements (High Availability, Scalability, Performance, Security, Cost Optimization), and trade-offs.

AWS Service	Why It Was Chosen	How It Meets the 5 Requirements	Design Trade-offs or Challenges
Amazon Route 53	Chosen for its highly reliable and scalable DNS service as the entry point for global user access.	High Availability: Directs traffic away from unhealthy endpoints with health checks. Scalability: Handles global queries seamlessly. Performance: Latency-based routing to nearest endpoints. Security: DNSSEC support. Cost Optimization: Pay-per-query model.	Management Complexity: Configuring advanced routing policies can be intricate for global setups.
Amazon CloudFront	Chosen to deliver static content and 3D assets quickly to a global audience via CDN.	High Availability: Global edge locations ensure redundancy. Scalability: Auto-handles traffic spikes. Performance: Edge caching for fast loads and 3D rendering. Security: Integrates with WAF. Cost Optimization: Reduces origin load, pay for transfers.	Cache Invalidation: Frequent updates may incur delays or extra costs.
AWS WAF	Chosen as the first line of defense against web exploits at the edge.	High Availability: Always-on protection. Scalability: Scales with traffic. Performance: Filters bad requests early. Security: Blocks SQL injection, XSS, bots. Cost Optimization: Prevents wasteful backend processing.	Configuration Complexity: Rules may block legitimate traffic if misconfigured.
Amazon S3	Chosen for durable, scalable storage of 3D assets and static files.	High Availability: 99.999999999% durability. Scalability: Unlimited storage. Performance: Fast access via CloudFront. Security: Encryption at rest/transit. Cost Optimization: Tiered storage (e.g., Intelligent-Tiering).	Eventual Consistency: May delay for rapidly updated objects.
Amazon API Gateway	Chosen as a managed front door for secure API management.	High Availability: Multi-AZ by default. Scalability: Handles millions of calls. Performance: Caching and throttling. Security:	Cost: High-volume APIs can accumulate fees.

AWS Service	Why It Was Chosen	How It Meets the 5 Requirements	Design Trade-offs or Challenges
		Integrates with Cognito/WAF. Cost Optimization: Pay-per-request.	
Amazon Cognito	Chosen for managed user authentication and authorization.	High Availability: Global replication. Scalability: Supports millions of users. Performance: Quick token issuance. Security: Secure sign-in, MFA. Cost Optimization: Tiered MAU pricing.	Customization: Non-standard flows require extra effort.
AWS Lambda	Chosen for serverless, event-driven backend logic (e.g., order processing).	High Availability: Multi-AZ execution. Scalability: Auto-scales to thousands. Performance: Fast invocation for dynamic requests. Security: IAM roles. Cost Optimization: Pay-per-use, no idle costs.	Cold Starts: Initial latency on infrequent calls.
Amazon DynamoDB	Chosen for low-latency, scalable NoSQL storage of dynamic data like carts.	High Availability: Multi-AZ replication. Scalability: On-demand mode. Performance: Millisecond reads. Security: Encryption, fine-grained access. Cost Optimization: Provisioned vs. on-demand.	Data Model: No complex SQL joins.
Amazon DAX	Chosen as in-memory cache for DynamoDB to boost read performance.	High Availability: Multi-node clusters. Scalability: Auto-scales. Performance: Sub-ms latency. Security: VPC-integrated. Cost Optimization: Reduces DynamoDB reads.	Management: Additional service to monitor.
Amazon RDS (PostgreSQL Multi-AZ)	Chosen for relational data with integrity (e.g., product catalogs).	High Availability: Automatic failover. Scalability: Read replicas. Performance: Optimized queries. Security: Encrypted backups. Cost Optimization: Reserved instances.	Operational Overhead: More management than serverless.
Amazon ElastiCache (Redis)	Chosen for session caching and query acceleration.	High Availability: Multi-AZ clusters. Scalability: Node scaling. Performance: In-memory speed. Security: Encryption. Cost Optimization: Reduces database load.	Cost: Hourly node fees even if idle.
Amazon EC2 & Auto Scaling Group	Chosen for compute-heavy 3D rendering tasks.	High Availability: ASG replaces failed instances. Scalability: Demand-based scaling. Performance: Dedicated CPUs/GPUs. Security: Security groups. Cost Optimization: Scale down off-peak.	Management: More overhead than Lambda.
Application Load Balancer (ALB)	Chosen to distribute traffic to EC2 instances.	High Availability: Multi-AZ. Scalability: Handles load. Performance: Path-based routing. Security: HTTPS termination. Cost Optimization: LCU-based.	Cost: Fixed hourly fee.
Amazon VPC & Subnets	Chosen for secure, isolated networking.	High Availability: Multi-AZ subnets. Scalability: Flexible CIDR. Performance: Low-latency internal traffic. Security: Private isolation. Cost Optimization: VPC Endpoints save on transfers.	Network Complexity: Setup of routing/security groups.
Amazon CloudWatch	Chosen for comprehensive monitoring and logging.	High Availability: Reliable metrics. Scalability: Handles all services. Performance: Real-time insights. Security: Log encryption. Cost Optimization: Identifies savings.	Cost: High-volume logs/metrics add up.
AWS Trusted Advisor	Chosen for automated best practice recommendations.	High Availability: Fault tolerance advice. Scalability: Scaling tips. Performance: Optimization suggestions. Security: Vulnerability checks. Cost Optimization: Cost-saving alerts.	Actionability: Recommendations need manual follow-up